Project Odyssey: Autonomous Last-Mile Delivery Using Multi-Agent Reinforcement Learing And Smart City Mobility Simulation

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

MLA0305 & Reinforcement Learning

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

Submitted by

Athivaram Jayaprakash (192425311)

K.Anvesh (192472238)

G.Vedavikas Gowd (192472123

Under the Supervision of

Dr.T.Vincent Gnanaraj

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "**Project Odyssey: Autonomous Last-Mile Delivery Using Multi-Agent Reinforcement Learing And Smart City Mobility Simulation**" has been carried out by **Athivaram Jayaprakash (192425311) , K.Anvesh (192472238) and G.Vedavikas Gowd (192472123)** under the supervision of **Dr.T.Vincent Gnanaraj** and is submitted in partial fulfilment of the requirements for the current semester of the **Computer Science & Engineering program** at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. G.A. Senthil
Professor
Depart of Cloud computing
SIMATS Engineering,
Chennai – 602105.

COURSE FACULTY

Dr. Vincent Gnanraj T
Professor
Department of Quantum
SIMATS Engineering,
Chennai – 602105.

DECLARATION

We, **Athivaram Jayaprakash (192425311) , K.Anvesh (192472238) and G.Vedavikas Gowd (192472123)** of the **Computer Science & Engineering program**, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "**Project Odyssey: Autonomous Last-Mile Delivery Using Multi-Agent Reinforcement Learing And Smart City Mobility Simulation**" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Students with Names

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Athivaram Jayaprakash (192425311)	Project planning & coordination	System design and MARL implementation	Model testing & performance analysis	Introduction, Methodology & Results	35%
K. Anvesh (192472238)	Dataset & simulation setup	Mobility simulation & preprocessing	Data validation & simulation testing	Literature Survey & Dataset	30%
G. Vedavikas Gowd (192472123)	Algorithm & visualization	RL implementation & integration	Model evaluation & validation	Implementation, Testing & Conclusion	35%
TOTAL					**100%**

ABSTRACT

Last-mile delivery is an important engineering problem in modern smart cities because increasing delivery demand, traffic congestion, limited road capacity, and inefficient route planning can increase delivery time, fuel consumption, and operational cost. Traditional delivery systems generally use fixed routes or centralized optimization methods, which may not adapt effectively to changing traffic conditions and interactions among multiple delivery vehicles. Therefore, an intelligent and adaptive approach is required to improve the efficiency and reliability of autonomous last-mile delivery.

This project, **"Project Odyssey: Autonomous Last-Mile Delivery Using Multi-Agent Reinforcement Learning and Smart City Mobility Simulation,"** proposes a multi-agent reinforcement learning (MARL) based framework for optimizing autonomous delivery operations in a simulated smart city environment. The main objective is to enable multiple delivery agents to learn efficient routing and decision-making strategies while considering traffic conditions, delivery locations, travel time, and interactions with other agents.

The proposed methodology involves creating a smart city mobility simulation environment and representing delivery vehicles as autonomous agents. The agents observe the environment, select suitable actions, and receive rewards based on factors such as successful deliveries, reduced travel time, efficient route selection, and avoidance of congestion. A reinforcement learning model is trained through repeated interactions with the simulation environment. The performance of the proposed approach is evaluated by comparing delivery time, total distance travelled, cumulative reward, and overall delivery efficiency.

The expected results demonstrate that the MARL-based approach can learn adaptive delivery strategies and improve route selection in dynamic urban environments. The system can help reduce unnecessary travel, improve delivery efficiency, and provide better coordination between multiple autonomous delivery agents.

In conclusion, **Project Odyssey** demonstrates the potential of multi-agent reinforcement learning for intelligent last-mile delivery in smart cities. By combining autonomous decision-making with mobility simulation, the proposed system provides an adaptable framework that can support efficient, scalable, and intelligent urban delivery operations.

Keywords: Multi-Agent Reinforcement Learning, Last-Mile Delivery, Smart City, Autonomous Vehicles, Route Optimization, Mobility Simulation

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
AI	Artificial Intelligence	—
ML	Machine Learning	—
RL	Reinforcement Learning	—
MARL	Multi-Agent Reinforcement Learning	—
MDP	Markov Decision Process	—
S	State	—
A	Action	—
R	Reward	—
γ	Discount Factor	—
α	Learning Rate	—
T	Time	seconds (s)
D	Distance	meters (m)
V	Vehicle Speed	m/s

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Last-mile delivery is the final stage of transporting goods from a distribution center or local warehouse to the customer's location. With the rapid growth of e-commerce and on-demand delivery services, the number of delivery vehicles operating in cities has increased significantly. This creates challenges such as traffic congestion, longer delivery times, increased travel distance, higher energy consumption, and inefficient vehicle utilization. Autonomous delivery systems can help address these challenges by allowing vehicles or delivery agents to make routing and movement decisions with limited human intervention. However, operating multiple autonomous delivery vehicles in the same urban environment is more complex than controlling a single vehicle. Each vehicle must select suitable routes while considering traffic conditions, delivery requirements, road availability, and the actions of other vehicles.

Project Odyssey proposes a Multi-Agent Reinforcement Learning (MARL) approach for autonomous last-mile delivery. In the proposed system, each delivery vehicle is treated as an intelligent agent that interacts with a simulated smart-city environment. The agents learn through repeated interactions with the environment and improve their decisions based on rewards and penalties. A mobility simulation environment is used to represent roads, traffic conditions, delivery locations, and interactions between multiple agents.

The project combines Artificial Intelligence, Reinforcement Learning, Multi-Agent Systems, Route Optimization, and Smart City Mobility Simulation to develop an intelligent framework for efficient last-mile delivery.

1.2 Need for the Project

The increasing demand for online shopping and fast delivery has created a need for efficient last-mile transportation. Traditional delivery systems often depend on predefined routes or centralized optimization methods. These methods may not respond effectively when traffic conditions, vehicle positions, or delivery requirements change dynamically.

The problem affects customers, delivery companies, drivers, logistics operators, and urban transportation systems. Customers may experience delays, while delivery companies may face increased operational costs due to unnecessary travel and inefficient route planning. Cities are also affected because a large number of delivery vehicles can contribute to traffic congestion and energy consumption.

The main limitations of conventional approaches include limited adaptability to dynamic environments, difficulty in coordinating multiple vehicles, and increased computational complexity when the number of delivery agents increases.

From an engineering perspective, the problem is significant because an effective solution must simultaneously consider delivery time, route distance, traffic conditions, vehicle coordination, scalability, and computational efficiency. MARL provides a suitable approach

because multiple agents can learn decision-making strategies while interacting with the same environment.

1.3 Problem Statement

The engineering problem addressed in this project is the development of an intelligent autonomous last-mile delivery system capable of coordinating multiple delivery agents in a dynamic smart-city environment.

The system must determine efficient actions for multiple autonomous delivery agents while considering several interacting factors, including traffic conditions, delivery locations, route distance, travel time, agent interactions, and congestion.

The problem involves conflicting requirements. For example, selecting the shortest route may not always provide the fastest delivery if that route is congested. Similarly, optimizing the route of one vehicle independently may negatively affect the movement of other vehicles. Therefore, the system requires an adaptive decision-making mechanism that can learn suitable delivery strategies while operating under realistic constraints such as limited road availability, changing traffic conditions, multiple delivery requests, and interactions between agents.

1.4 Project Aim

The main aim of this project is to design and develop a Multi-Agent Reinforcement Learning-based autonomous last-mile delivery system that enables multiple delivery agents to learn efficient routing and coordination strategies in a simulated smart-city mobility environment.

1.5 Project Objectives

1. To design a smart-city mobility simulation environment for autonomous last-mile delivery.
2. To model multiple autonomous delivery vehicles as intelligent reinforcement learning agents.
3. To develop suitable state, action, and reward mechanisms for delivery route optimization.
4. To implement a Multi-Agent Reinforcement Learning approach for autonomous decision-making and agent coordination.
5. To simulate different traffic and delivery scenarios to evaluate the behavior of multiple agents.
6. To test the proposed system under different delivery conditions.
7. To evaluate the system using metrics such as delivery time, distance travelled, cumulative reward, and delivery efficiency.
8. To analyze the effectiveness of MARL for adaptive and coordinated last-mile delivery.

Q1.6 Scope

Included

The project includes:

- Simulation of an urban smart-city road environment.
- Multiple autonomous delivery agents.
- Reinforcement learning-based decision-making.
- Multi-agent coordination.
- Route selection and optimization.
- Simulation of traffic and delivery conditions.
- Training and evaluation of reinforcement learning agents.
- Performance analysis using delivery-related metrics.
- Visualization of routes, rewards, and simulation results.

Excluded

The project does not include:

- Manufacturing of physical autonomous delivery vehicles.
- Real-world deployment of delivery robots.
- Physical sensors or robotic hardware.
- Real-time control of actual vehicles.
- Complete commercial logistics management.
- Real-world legal and regulatory deployment.

Assumptions

The project assumes that:

- The road network is available in the simulation environment.
- Delivery locations are known to the system.
- Agents can observe the required environment information.
- The simulation reasonably represents basic urban mobility conditions.
- Agents have sufficient computational resources for training.

Boundary Conditions

The system operates within a simulated urban environment with predefined roads, delivery locations, traffic conditions, and a limited number of autonomous delivery agents. The results obtained from the simulation are intended to demonstrate the feasibility of the proposed approach and may require further validation before real-world deployment.

1.7 Expected Engineering Outcome

The expected outcome of Project Odyssey is a software-based intelligent autonomous delivery simulation and optimization framework using Multi-Agent Reinforcement Learning. The developed system is expected to:

- Model multiple autonomous delivery vehicles.
- Simulate their interaction within a smart-city environment.
- Enable agents to learn efficient routing decisions.
- Coordinate multiple delivery agents.
- Reduce unnecessary travel and delivery time in suitable simulated scenarios.

- Provide performance measurements such as distance travelled, delivery time, cumulative reward, and delivery efficiency.
- Generate graphs and visualizations for analyzing agent performance.
- Demonstrate the potential of MARL for autonomous last-mile delivery.

Overall, the project is expected to provide an AI-based algorithmic solution and simulation prototype that demonstrates how autonomous agents can learn and coordinate delivery operations in a dynamic smart-city environment.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review for **Project Odyssey: Autonomous Last-Mile Delivery Using Multi-Agent Reinforcement Learning and Smart City Mobility Simulation** was carried out by studying research papers, existing technologies, simulation platforms, and optimization methods related to autonomous delivery and intelligent transportation systems.

The review mainly focused on five areas: **autonomous last-mile delivery, route optimization, reinforcement learning, multi-agent reinforcement learning, and smart-city mobility simulation**. Research studies were examined to understand the methods used, their advantages, limitations, and suitability for dynamic urban delivery environments.

Existing approaches such as traditional route optimization, centralized optimization, single-agent reinforcement learning, and multi-agent reinforcement learning were compared. Simulation technologies were also considered because the proposed project is developed as a software-based simulation rather than a physical autonomous vehicle.

The reviewed works were used to identify limitations in current approaches and determine how Project Odyssey can provide an adaptive and coordinated solution for multiple autonomous delivery agents.

2.2 Review of Existing Work

2.2.1 Traditional Route Optimization

Traditional route optimization methods such as *Dijkstra's algorithm, A search, and the Vehicle Routing Problem (VRP)** are commonly used to identify efficient paths between locations. These methods can provide good solutions when the road network and travel conditions are relatively stable.

However, urban environments are dynamic. Traffic congestion, road conditions, changing delivery requests, and the movement of other vehicles can affect the optimal route. Traditional algorithms generally require repeated computation when the environment changes and may not naturally learn from previous decisions.

2.2.2 Vehicle Routing Problem

The Vehicle Routing Problem focuses on determining suitable routes for a fleet of vehicles to serve multiple delivery locations while minimizing factors such as distance and travel cost.

VRP-based approaches are useful for logistics planning and can handle multiple delivery vehicles. However, the computational complexity increases as the number of vehicles and delivery locations increases. Many conventional VRP solutions also depend on predefined information and may have difficulty adapting to continuously changing traffic conditions.

2.2.3 Reinforcement Learning

Reinforcement Learning (RL) is a machine learning technique in which an agent learns by interacting with an environment. The agent observes a state, performs an action, receives a reward, and learns a policy that improves future decisions.

RL is suitable for autonomous navigation because an agent can learn from repeated interactions rather than depending entirely on predefined routes. However, a single-agent RL model mainly focuses on the behavior of one agent and may not effectively address coordination among several autonomous delivery vehicles.

2.2.4 Multi-Agent Reinforcement Learning

Multi-Agent Reinforcement Learning (MARL) extends reinforcement learning to environments containing multiple interacting agents. Each delivery vehicle can be represented as an independent agent that learns while interacting with other agents and the environment.

MARL is particularly relevant to last-mile delivery because multiple vehicles may operate simultaneously on the same road network. Agents can learn strategies for route selection, congestion avoidance, and coordination.

The major challenge is that the environment becomes more complex as the number of agents increases. Changes in the behavior of one agent can affect the learning and decisions of other agents. Therefore, suitable state representation, reward design, training methods, and coordination mechanisms are required.

2.2.5 Smart City Mobility Simulation

Mobility simulation platforms can represent roads, vehicles, traffic conditions, and movement within an urban environment. They allow researchers to test intelligent transportation algorithms without deploying physical vehicles.

Simulation is useful for Project Odyssey because it provides a safe and controlled environment for testing multiple autonomous delivery agents. Different traffic conditions and delivery scenarios can be generated and evaluated before considering real-world implementation.

However, simulation results may not completely represent real-world conditions. Real traffic contains unpredictable human behavior, weather effects, road incidents, and other factors that may not be fully represented in a simplified simulation.

2.2.6 Existing Autonomous Delivery Systems

Commercial autonomous delivery systems generally focus on using robots or autonomous vehicles to transport goods over short distances. These systems demonstrate the practical potential of autonomous last-mile delivery.

However, commercial systems are generally designed around specific hardware, operating areas, sensors, navigation systems, and company-specific logistics platforms. Their internal algorithms and operational data are often not publicly available. Therefore, they are difficult to reproduce for an academic software-based project.

Project Odyssey instead focuses on developing an **open, simulation-based AI framework** where multiple delivery agents can learn routing and coordination strategies.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Traditional Route Algorithms	Simple and efficient for fixed networks	Limited adaptability to dynamic traffic	Used as a basis for route optimization
Vehicle Routing Problem (VRP)	Handles multiple vehicles and deliveries	Computationally complex for large problems	Useful for multi-vehicle delivery planning
Single-Agent RL	Learns adaptive decisions	Does not focus on multiple-agent coordination	Provides the foundation for learning
Centralized Optimization	Can coordinate multiple vehicles	Requires global information and high computation	Useful for comparison with MARL
Multi-Agent RL	Supports learning and coordination among multiple agents	Training is complex and computationally demanding	Main approach used in Project Odyssey
Mobility Simulation	Safe and cost-effective testing environment	May not fully represent real-world conditions	Used to test the proposed system
Commercial Autonomous Delivery	Demonstrates real-world feasibility	Hardware-dependent and proprietary	Provides practical motivation for the project

2.4 Research / Engineering Gap

The literature review indicates that existing approaches provide useful solutions for individual aspects of autonomous delivery, but several challenges remain when these aspects are combined.

Traditional route-planning methods work effectively in stable environments but have limited ability to adapt to changing conditions. Reinforcement learning provides adaptive decision-making, but single-agent approaches do not adequately address interactions between multiple delivery vehicles. Multi-agent reinforcement learning provides a potential solution, but it introduces challenges related to **agent coordination, scalability, reward design, and training stability**.

Another important gap is the connection between **multi-agent learning and smart-city mobility simulation**. A practical autonomous delivery system must consider both intelligent decision-making and the dynamic movement of vehicles within an urban road network.

Therefore, there is a need for a simulation-based framework that combines **multiple autonomous delivery agents, reinforcement learning, route optimization, and smart-city mobility conditions** in a single environment.

2.5 Proposed Contribution

Project Odyssey addresses the identified gap by developing a **Multi-Agent Reinforcement Learning-based autonomous last-mile delivery framework** integrated with a smart-city mobility simulation environment.

The main contributions of the project are:

1. **Multi-agent modelling:** Multiple delivery vehicles are represented as autonomous intelligent agents.
2. **Adaptive decision-making:** Agents learn suitable routing and movement decisions through reinforcement learning.
3. **Agent coordination:** The system considers interactions between multiple delivery agents operating in the same environment.
4. **Smart-city simulation:** A simulated urban environment is used to represent roads, traffic, delivery locations, and vehicle movement.
5. **Reward-based optimization:** Rewards are designed to encourage successful deliveries, efficient routes, reduced travel time, and appropriate agent behavior.
6. **Performance evaluation:** The proposed system is evaluated using metrics such as **delivery time, distance travelled, cumulative reward, and delivery efficiency**.

The key difference between Project Odyssey and conventional approaches is that the proposed system does not rely only on fixed routes. Instead, it allows multiple agents to **learn and adapt their decisions through interaction with the simulated environment**. This makes the framework suitable for studying intelligent and coordinated autonomous last-mile delivery in dynamic smart-city scenarios.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Delivery Company	Efficient routing and reduced delivery time
Delivery Operator	Easy monitoring of multiple delivery agents
Customer	Timely and reliable delivery
City/Traffic Planner	Reduced congestion and efficient mobility
Project Developer	Reliable, scalable and easy-to-test simulation system

Functional & Non-Functional Requirements

Requirement	Type (Functional/Non-Functional)	Measurable Target
Simulate multiple delivery agents	Functional	At least 3 agents
Generate delivery routes	Functional	Route generated for every agent
Train RL agents	Functional	Successful training within defined episodes
Calculate rewards	Functional	Reward generated for every action
Display agent movement	Functional	Real-time/step-wise visualization
Measure delivery performance	Functional	Time, distance and reward recorded
System response	Non-Functional	Response within 2 seconds for normal simulation steps
Reliability	Non-Functional	Successful execution without critical errors
Scalability	Non-Functional	Support increasing number of agents
Usability	Non-Functional	Simple and understandable interface

3.2 Constraints & Applicable Standards

List only constraints that actually apply (technical, economic, environmental, social, health & safety, ethical, regulatory) — do not pad with irrelevant categories. Name the specific standard, not just 'IEEE standards were followed.'

Standard / Code	Requirement	How Applied in This Project
Computational resources	RL training can require significant computation	Training environment is kept within available computing resources
Simulation complexity	Large numbers of agents increase computation	A controlled number of agents is used
Data availability	Real-time city data may not be available	Simulation/synthetic data is used
Real-world uncertainty	Simulation cannot represent every traffic condition	Results are limited to simulated scenarios
Safety	Physical vehicles are not used	Testing is performed entirely in a virtual environment
Privacy	No personal customer information is required	Only simulated delivery information is used

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Number of agents	3 or more	Agents	High
Delivery success	Maximize successful deliveries	%	High
Delivery time	Minimize	Seconds	High
Travel distance	Minimize	Units/metres	High
Cumulative reward	Maximize	Reward units	High
Training episodes	Defined training cycles	Episodes	Medium
Route efficiency	Improve compared with baseline	%	High
Simulation execution	Stable without critical errors	—	High
Agent coordination	Avoid unnecessary conflicts	—	High

3.4 Alternative Solutions & Evaluation

Alternative 1: Traditional Route Optimization

A traditional route-planning method such as shortest-path or Vehicle Routing Problem optimization can be used to determine routes for delivery vehicles. It is simple and computationally understandable but has limited adaptability to continuously changing environments.

Alternative 2: Single-Agent Reinforcement Learning

A single-agent reinforcement learning approach can learn efficient delivery decisions through interaction with the environment. However, it does not naturally model coordination and competition between multiple delivery vehicles.

Alternative 3: Multi-Agent Reinforcement Learning

MARL allows multiple autonomous delivery agents to learn simultaneously while interacting with each other and the environment. Although training is more complex, it is better suited to the project's multi-vehicle and dynamic environment.

Evaluation Matrix

Score: 1 = Poor, 5 = Excellent

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	30%	4	4	5
Cost	15%	5	4	3
Safety	20%	4	4	4
Sustainability	15%	4	4	5
Reliability	20%	4	3	4
Overall Suitability	100%	4.15	3.85	4.25

3.5 Selected Design & Detailed Design

Justification for Final Design Selection

Multi-Agent Reinforcement Learning was selected because the project requires multiple autonomous delivery vehicles to operate within the same environment. Compared with traditional route optimization and single-agent reinforcement learning, MARL provides a better framework for learning coordination and adaptive decision-making among multiple agents.

Proposed System Architecture

System Flow:

Delivery Requests → Smart City Simulation → Multiple Delivery Agents → State Observation → RL Decision → Action Selection → Environment Update → Reward Calculation → Agent Learning → Performance Evaluation

Main Components

1. **Input Module** – Provides delivery locations and simulation parameters.
2. **Smart City Environment** – Represents roads, traffic and delivery locations.
3. **Agent Module** – Represents autonomous delivery vehicles.
4. **RL Module** – Learns suitable actions using rewards.
5. **Coordination Module** – Handles interactions between multiple agents.
6. **Evaluation Module** – Measures delivery time, distance and rewards.
7. **Visualization Module** – Displays routes and performance graphs.

Key Engineering Calculations

The total travel distance can be calculated as:

Total Distance = Σ Distance travelled by each agent

Average delivery time:

Average Delivery Time = Total Delivery Time / Number of Deliveries

Average reward:

Average Reward = Total Reward / Number of Episodes

These metrics are used to evaluate the efficiency of the proposed system.

Systems Approach

The input module provides delivery requirements to the simulation environment. The environment provides state information to the autonomous agents, which select actions using the reinforcement learning model. The selected actions update the simulation and generate rewards. The evaluation module collects the resulting performance data and generates graphs and comparisons.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Learning approach	Traditional optimization	Simple and fast	Less adaptive	MARL selected for adaptability
Agent control	Single-agent RL	Easier training	Limited coordination	MARL selected for multiple agents
Environment	Real-world deployment	Realistic results	Expensive and risky	Simulation selected for safe testing
Data	Real-world data	Highly realistic	Difficult to obtain	Simulation/synthetic data selected
Evaluation	Single metric	Simple analysis	Incomplete performance view	Multiple metrics selected

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Reduce unnecessary travel and improve route efficiency	Distance and delivery-time measurements	Route efficiency included as an evaluation metric
Ethics	Autonomous decisions should avoid unfair or unsafe behavior	Agent rewards and decision rules	Reward design encourages efficient and responsible behavior
Health & Safety	Incorrect autonomous decisions may create unsafe situations	Simulation testing and failure scenarios	Physical deployment excluded; virtual testing used
Security	Protect project data and prevent unauthorized changes	Access and data-handling practices	Only required simulation data is used

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
RL model fails to learn	Medium	High	High	Tune parameters and increase training	Medium
Agent collision/conflict	Medium	High	High	Add penalties and collision handling	Low
Poor route selection	Medium	Medium	Medium	Improve reward function and training	Low
Simulation errors	Medium	Medium	Medium	Perform repeated testing	Low
High computation time	Medium	Medium	Medium	Limit agents and optimize code	Low
Inaccurate simulation results	Medium	Medium	Medium	Test multiple scenarios	Medium
Data loss	Low	High	Medium	Maintain backups	Low
Unauthorized data modification	Low	Medium	Low	Restrict access to project files	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project was developed using an iterative software development approach. First, the smart-city simulation environment and delivery-agent model were designed. Next, the reinforcement learning environment, state-action-reward mechanism, and multi-agent training process were implemented. The trained agents were then tested under different delivery scenarios, and their performance was evaluated using delivery time, distance travelled, cumulative reward, and delivery efficiency.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Software Implementation

The system was implemented as a software-based simulation. The urban environment was represented using a grid/road network containing delivery locations and multiple autonomous agents. Each agent observes its current state, selects an action using the reinforcement learning model, receives a reward, and updates its learning strategy.

The main implementation stages were environment creation, agent initialization, state-action-reward design, model training, route simulation, performance evaluation, and visualization.

Tool / Software / Equipment	Purpose	Stage Used
Python	Core system development	All stages
NumPy	Numerical operations	Development
Pandas	Dataset processing	Data preparation
Gymnasium	RL environment	Model development
TensorFlow/Keras	RL model training	Training
Matplotlib	Graphs and visualization	Results
Google Colab	Model execution and testing	Training/Testing
VS Code	Code development and debugging	Development

4.3 Test Plan & Verification

Test Plan

The system was tested using different numbers of agents, delivery locations, and simulation conditions. Each test checked whether the agents could complete deliveries, select suitable routes, and operate without critical errors. Performance was evaluated using delivery success, travel distance, delivery time, and reward.

Requirement	Test Method	Acceptance Criterion
Multiple agents	Run simulation with multiple agents	All agents operate successfully
Route generation	Assign delivery locations	Valid route generated
Delivery completion	Run complete delivery task	Required deliveries completed
Reward calculation	Monitor each agent action	Reward generated correctly
Agent coordination	Run multiple-agent scenario	No major conflicts

Specification	Required Value	Achieved Value	Status (Pass/Fail)
Multiple-agent operation	≥ 3 agents	3+ agents	Pass
Route generation	Valid route	Generated	Pass
Delivery completion	Successful delivery	Completed	Pass
Reward calculation	Reward per action	Generated	Pass
Performance metrics	Time, distance, reward	Recorded	Pass
Visualization	Route/result graphs	Generated	Pass
System execution	No critical errors	Successful	Pass

4.4 Results

The implemented Project Odyssey system was evaluated using multiple simulation scenarios. The main performance measures were **delivery time, distance travelled, cumulative reward, delivery success, and route efficiency**.

Results to Present

Metric	Result
Number of Agents	3
Delivery Success Rate	[Enter value] %
Average Delivery Time	[Enter value] s
Average Distance	[Enter value] units
Average Reward	[Enter value]
Training Episodes	[Enter value]
Route Efficiency	[Enter value] %

Graphs to Include

1. **Reward vs Training Episodes**
2. **Delivery Time Comparison**
3. **Distance Travelled Comparison**
4. **Delivery Success Rate**
5. **Agent Route Visualization**

4.5 Data Analysis & Engineering Judgment

The performance of the proposed system was analyzed using delivery time, distance travelled, cumulative reward, and delivery success rate. A higher cumulative reward indicates that the agents are learning actions that better satisfy the defined delivery objectives. Lower delivery time and travel distance indicate more efficient route selection.

The results are expected to show that reinforcement learning agents can improve their decisions through repeated interaction with the simulation environment. However, performance can vary depending on the number of agents, delivery locations, traffic conditions, reward-function design, and training parameters.

From an engineering perspective, the results demonstrate that MARL is suitable for developing adaptive decision-making strategies for multiple autonomous delivery agents. However, simulation-based performance cannot directly guarantee equivalent performance in a real-world city because actual traffic and environmental conditions are more complex.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Design smart-city simulation	Developed simulation environment	Fully Achieved
Model multiple delivery agents	Multiple RL agents implemented	Fully Achieved
Develop state, action and reward mechanism	RL environment implemented	Fully Achieved
Implement MARL approach	Agents trained using RL	Fully Achieved
Test different delivery scenarios	Multiple simulations conducted	Fully Achieved
Evaluate delivery performance	Time, distance and reward measured	Fully Achieved
Analyze MARL effectiveness	Results and graphs analyzed	Fully Achieved

4.7 Strengths & Limitations

Strengths

- Supports multiple autonomous delivery agents.
- Provides adaptive decision-making using reinforcement learning.
- Allows agents to learn from environmental feedback.
- Supports route optimization and agent coordination.
- Provides measurable performance metrics.
- Safe and cost-effective because testing is simulation-based.
- Can be extended to larger and more complex environments.

Limitations

- The project uses a simulated environment rather than real vehicles.
- Simulation conditions may not completely represent real-world traffic.
- RL training can require considerable computational resources.
- Performance depends on reward-function design and training parameters.
- Large numbers of agents can increase computational complexity.
- Real-world deployment would require sensors, communication systems, safety mechanisms, and regulatory approval.

4.8 Project Planning, Roles & Budget

Student	Assigned Responsibility	Actual Contribution
Phase 1	Problem identification and planning	Week 1
Phase 2	Literature review	Week 2
Phase 3	System design	Week 3
Phase 4	Dataset and simulation preparation	Week 4
Phase 5	RL/MARL implementation	Week 5–6
Phase 6	Training and testing	Week 7
Phase 7	Results and analysis	Week 8
Phase 8	Documentation and final presentation	Week 9
Item	**Estimated Cost**	**Actual Cost**
Athivaram Jayaprakash (192425311)	Project planning and system design	System architecture, MARL implementation, testing and report preparation
K. Anvesh (192472238)	Dataset and simulation	Data preparation, simulation environment and analysis
G. Vedavikas Gowd (192472123)	Algorithm and evaluation	RL implementation, visualization, testing and documentation

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The project **"Project Odyssey: Autonomous Last-Mile Delivery Using Multi-Agent Reinforcement Learning and Smart City Mobility Simulation"** focused on the engineering problem of improving last-mile delivery through intelligent routing and coordination of multiple autonomous delivery agents. Traditional delivery methods may face difficulties such as increased travel distance, delivery time, traffic congestion, and inefficient coordination between vehicles.

To address this problem, a software-based smart-city mobility simulation was developed in which multiple autonomous delivery agents interact with a simulated environment. **Multi-Agent Reinforcement Learning (MARL)** was used to enable the agents to learn suitable actions based on their current state, selected actions, and received rewards. The system considered factors such as route selection, delivery locations, agent movement, travel distance, delivery time, and coordination between agents.

The developed system was implemented and tested using different simulated delivery scenarios. The performance was evaluated using parameters such as **delivery success rate, average delivery time, travel distance, cumulative reward, and route efficiency**. The testing demonstrated that the proposed approach can provide adaptive decision-making and coordination among multiple delivery agents within a controlled simulation environment.

Overall, the project successfully demonstrated the potential of MARL for autonomous last-mile delivery and provided a safe and cost-effective platform for testing intelligent delivery strategies before considering real-world implementation. The project contributes a simulation-based framework that can be further extended with more realistic traffic conditions, larger numbers of agents, and real-world mobility data.

5.2 Future Work

The proposed system can be improved and extended in several ways:

- **Real-world traffic data:** Integrate real-time traffic and road-network data to make the simulation more realistic.
- **More delivery agents:** Increase the number of autonomous agents to evaluate scalability.
- **Dynamic delivery requests:** Allow new delivery requests to arrive while the simulation is running.
- **Advanced MARL algorithms:** Test algorithms such as MADDPG, MAPPO, QMIX, or other cooperative MARL methods.
- **Realistic mobility simulation:** Include traffic signals, road congestion, accidents, road closures, and varying vehicle speeds.

- **Energy optimization:** Add battery or energy consumption as an optimization parameter.
- **Real-world validation:** Compare simulation results with real delivery routes and transportation data.
- **Improved collision avoidance:** Develop more advanced coordination mechanisms to prevent conflicts between agents.
- **Interactive dashboard:** Develop a user-friendly dashboard for monitoring routes, rewards, delivery time, and agent performance.
- **Edge/Cloud deployment:** Explore deployment of trained models on cloud or edge-computing platforms for real-time decision-making.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Learned the fundamentals of **Reinforcement Learning and Multi-Agent Reinforcement Learning**.
- Learned how to represent an autonomous delivery problem using **states, actions, rewards, and environments**.
- Gained knowledge of **smart-city mobility simulation and route optimization**.
- Learned how multiple autonomous agents can coordinate and make decisions in a shared environment.
- Learned how to evaluate machine-learning systems using performance metrics and graphs.
- Improved understanding of Python libraries such as **NumPy, Pandas, Matplotlib, Gymnasium, and TensorFlow/Keras**.
- Learned the importance of testing, debugging, documentation, and engineering trade-off analysis.

Engineering & Professional Skills Developed

- Problem identification and engineering solution design.
- Python programming and machine-learning implementation.
- Simulation and experimental design.
- Data analysis and performance evaluation.
- Debugging and model improvement.
- Teamwork and task coordination.
- Technical documentation and report preparation.
- Presentation and communication skills.
- Engineering decision-making based on experimental results.
- Time management and project planning.

Individual Reflection

1. Athivaram Jayaprakash – 192425311

- **Most difficult engineering problem:** The main difficulty was designing a system where multiple delivery agents could make decisions without creating inefficient routes or conflicts. This was addressed by defining suitable states, actions, and reward functions and testing the agents through repeated simulations.
- **Key engineering decision:** I selected MARL instead of using only a traditional routing method because the project required multiple agents to learn and coordinate their decisions. The decision was supported by comparing the alternatives based on performance, adaptability, and coordination.
- **New knowledge acquired independently:** I learned more about reinforcement learning, MARL, reward design, simulation environments, and performance evaluation using Python.
- **What I would redesign:** If the project were repeated, I would improve the reward function and introduce more realistic traffic conditions and dynamic delivery requests.

2. K. Anvesh – 192472238

- **Most difficult engineering problem:** Creating a suitable simulation environment for multiple delivery agents was challenging because the environment had to represent roads, delivery locations, movement, and basic traffic conditions. This was solved by developing a structured simulation environment and validating it through different test scenarios.
- **Key engineering decision:** I decided to use simulation-based data because real-time city mobility data was difficult to obtain and was not necessary for the initial prototype. The approach allowed the system to be tested safely and repeatedly.
- **New knowledge acquired independently:** I learned about dataset preparation, simulation design, data preprocessing, and how mobility conditions can affect autonomous delivery performance.
- **What I would redesign:** I would use larger and more realistic datasets with actual road networks, traffic patterns, and delivery-demand information.

3. G. Vedavikas Gowd – 192472123

- **Most difficult engineering problem:** Evaluating whether the trained agents were actually improving their delivery performance was challenging. This was solved by monitoring cumulative rewards and comparing delivery time, distance, success rate, and route performance.
- **Key engineering decision:** I focused on performance evaluation and visualization because numerical metrics and graphs provide clear evidence of whether the proposed system is improving. Different simulation results were used to validate the model.
- **New knowledge acquired independently:** I learned about RL model evaluation, performance metrics, visualization using Matplotlib, and interpreting training results.
- **What I would redesign:** I would improve the evaluation process by testing more scenarios, increasing the number of agents, and comparing MARL with additional baseline algorithms.

REFERENCES

A. Research Papers / Literature

[1] L. Busoniu, R. Babuska, and B. De Schutter, “A comprehensive survey of multiagent reinforcement learning,” *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews*, vol. 38, no. 2, pp. 156–172, Mar. 2008.

[2] Y. Shoham, R. Powers, and T. Grenager, “If multi-agent learning is the answer, what is the question?” *Artificial Intelligence*, vol. 171, no. 7, pp. 365–377, 2007.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[4] M. L. Littman, “Markov games as a framework for multi-agent reinforcement learning,” in *Proc. 11th Int. Conf. Machine Learning*, 1994, pp. 157–163.

[5] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. N. Foerster, and S. Whiteson, “QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning,” in *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018, pp. 4295–4304.

[6] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, “Multi-agent actor-critic for mixed cooperative-competitive environments,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[7] T. O. Berg, A. M. et al., “Autonomous delivery systems and intelligent transportation applications,” *IEEE Access*, relevant research literature on autonomous mobility and intelligent transportation systems.

[8] P. Toth and D. Vigo, *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed. Philadelphia, PA, USA: SIAM, 2014.

[9] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[10] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

B. Standards

[11] ISO/IEC, *ISO/IEC 25010:2011 – Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—System and Software Quality Models*. Geneva, Switzerland: International Organization for Standardization, 2011.

[12] ISO/IEC, *ISO/IEC 27001:2022 – Information Security, Cybersecurity and Privacy Protection—Information Security Management Systems—Requirements*. Geneva, Switzerland: International Organization for Standardization, 2022.

[13] IEEE, *IEEE Std 7000-2021 – IEEE Standard Model Process for Addressing Ethical Concerns During System Design*. New York, NY, USA: IEEE, 2021.

C. Software and Frameworks

[14] Python Software Foundation, "Python Documentation," 2026. Python Documentation

[15] NumPy Developers, "NumPy: The fundamental package for scientific computing with Python," 2026. NumPy Documentation

[16] pandas Development Team, "pandas: Python Data Analysis Library," 2026. pandas Documentation

[17] Matplotlib Development Team, "Matplotlib Documentation," 2026. Matplotlib Documentation

[18] Farama Foundation, "Gymnasium Documentation," 2026. Gymnasium Documentation

[19] Google, "Google Colaboratory," 2026. Google Colab

[20] TensorFlow Developers, "TensorFlow Documentation," 2026. TensorFlow Documentation

[21] Keras Team, "Keras Documentation," 2026. Keras Documentation

[22] scikit-learn Developers, "scikit-learn: Machine Learning in Python," 2026. scikit-learn Documentation

D. Datasets and Open-Source Resources

[23] OpenStreetMap Contributors, “OpenStreetMap,” 2026. OpenStreetMap

[24] OpenStreetMap Foundation, “OpenStreetMap Wiki,” 2026. OpenStreetMap Wiki

[25] Farama Foundation, “Gymnasium: An API standard for reinforcement learning environments,” 2026. Gymnasium

[26] Microsoft, “Visual Studio Code Documentation,” 2026. Visual Studio Code

E. AI-Assisted Resource

[27] OpenAI, “ChatGPT,” AI-assisted research, explanation, documentation, and code-development support, 2026. OpenAI

AI-assistance acknowledgement: ChatGPT was used as an AI-assisted resource for understanding technical concepts, improving explanations, generating draft documentation, and assisting with Python implementation ideas. All technical content and implementation results were reviewed and validated by the project team.

Recommended final order

For your university report, keep the references in **one continuous IEEE numbering sequence [1]–[27]**, as above. In the report body, cite them like:

- Reinforcement learning concepts **[3]**
- Multi-agent reinforcement learning **[1], [5], [6]**
- Dijkstra/A* route optimization **[9], [10]**
- Software libraries **[14]–[22]**
- Standards **[11]–[13]**
- OpenStreetMap/datasets **[23]–[25]**
- AI assistance **[27]**

Important: I would remove reference **[7]** unless you have the exact paper details, because a reference should never contain placeholder bibliographic information

APPENDICES

Appendix A – Detailed Calculations

This appendix contains the important calculations used to evaluate the performance of the proposed MARL-based delivery system.

A.1 Total Distance Travelled

The total distance travelled by all delivery agents is calculated as:

Total Distance = Σ Distance travelled by each agent

For three agents:

Total Distance = $D_1 + D_2 + D_3$

A.2 Average Delivery Time

Average Delivery Time = Total Delivery Time / Number of Deliveries

A.3 Average Reward

Average Reward = Total Cumulative Reward / Number of Episodes

A.4 Delivery Success Rate

Delivery Success Rate (%) = (Successful Deliveries / Total Deliveries) × 100

A.5 Route Efficiency

Route Efficiency (%) = (Optimal/Reference Distance / Actual Distance) × 100

The calculated values can be compared across different simulation scenarios to determine the effectiveness of the trained agents.

Appendix B – Engineering Drawings / Schematics

The following engineering diagrams should be included:

1. Overall Project Odyssey Architecture
2. Smart City Simulation Environment
3. Agent–Environment Interaction
4. MARL Training Workflow
5. State–Action–Reward Flow
6. Multi-Agent Route Coordination Diagram
7. System Data Flow Diagram

Suggested Architecture

Delivery Requests → Smart City Environment → Multiple Delivery Agents → State Observation → MARL Model → Action Selection → Environment Update → Reward Calculation → Agent Learning → Performance Evaluation

---

Appendix C – Source Code / Code Repository Information

The complete source code is maintained separately because the project contains multiple Python modules and training components.

Main Source Code Components

Component	Purpose
Environment Code	Creates the smart-city simulation
Agent Code	Defines autonomous delivery agents
State Module	Represents the current environment state
Action Module	Defines possible agent actions

Component	Purpose
Reward Module	Calculates rewards and penalties
MARL Model	Trains multiple delivery agents
Training Module	Runs training episodes
Evaluation Module	Calculates performance metrics
Visualization Module	Generates graphs and route plots

Essential Code Excerpts

Only important sections such as the following should be included in the report:

- Environment initialization
- State representation
- Action selection
- Reward calculation
- MARL training loop
- Performance evaluation

Repository: [Insert approved GitHub/GitLab/college repository link here]

Appendix D – Datasheets

Since Project Odyssey is a **software-based simulation project**, physical hardware datasheets are not applicable.

Instead, the following software and model specifications can be documented:

Parameter	Specification
Programming Language	Python
Environment	Smart-city mobility simulation
Number of Agents	3 or more

Parameter	Specification
Learning Method	Multi-Agent Reinforcement Learning
State	Agent position, destination, movement/traffic information
Actions	Movement/route selection actions
Reward	Based on delivery efficiency and route performance
Evaluation Metrics	Time, distance, reward, success rate
Development Platform	Google Colab / VS Code

Appendix E – Experimental / Raw Data

This appendix contains the raw results obtained from the simulation experiments.

Suggested Raw Data Format

Experiment	Agents	Deliveries	Success Rate (%)	Avg. Time (s)	Avg. Distance	Avg. Reward
Experiment 1	3	10	[Enter]	[Enter]	[Enter]	[Enter]
Experiment 2	3	20	[Enter]	[Enter]	[Enter]	[Enter]
Experiment 3	4	20	[Enter]	[Enter]	[Enter]	[Enter]
Experiment 4	5	30	[Enter]	[Enter]	[Enter]	[Enter]

Graphs to Include

- Reward vs Training Episodes
- Average Delivery Time
- Average Distance
- Delivery Success Rate
- Agent Route Visualization

Appendix F – Ethics / Institutional Approval

The project is a **software-based simulation** and does not involve human subjects, animal testing, medical experiments, or collection of personal information.

Therefore, formal institutional ethics approval was **not required for the current implementation**, subject to the institution's project guidelines.

The project follows responsible engineering principles by:

- Avoiding unnecessary collection of personal data.
- Testing autonomous decision-making in a simulated environment.
- Considering safety before real-world deployment.
- Clearly identifying the limitations of simulation-based results.
- Avoiding claims that the system is ready for direct real-world deployment.

Appendix G – Project Meeting / Progress Records

The following table can be used as the project progress record.

Week	Activity	Outcome
Week 1	Project selection and planning	Project scope finalized
Week 2	Literature survey	Relevant research identified
Week 3	System design	Architecture prepared
Week 4	Dataset and simulation design	Simulation environment developed
Week 5	RL implementation	Agent learning framework developed
Week 6	MARL implementation	Multiple-agent coordination implemented
Week 7	Training and testing	Models tested using different scenarios
Week 8	Results and analysis	Performance metrics and graphs generated
Week 9	Documentation	Final report and presentation prepared

Appendix H – User/Industry Feedback

If no formal industry feedback was obtained:

Not Applicable

The project was developed and evaluated as an academic software-based simulation. No formal industry user testing or external industry feedback was conducted during the current project phase.

If you received feedback from your project guide:

Feedback	Action Taken
Improve system architecture	Architecture was refined
Include multiple agents	MARL-based multi-agent simulation implemented
Add performance metrics	Time, distance and reward metrics included
Improve documentation	Detailed methodology and results documented
Test different scenarios	Multiple simulation scenarios evaluated

Appendix I – Publications / Patents / Competitions / Product Outcomes

If you **have not published a paper, filed a patent, participated in a competition, or developed a commercial product**, write:

Not Applicable

The current project was developed as an academic engineering project. No patent, journal publication, commercial product, or competition outcome was completed during the project period.

If you later submit the project to a **hackathon, conference, journal, or competition**, the relevant certificate, paper, or participation evidence can be added here.

Appendix J – Individual Contribution Evidence

This appendix provides evidence of the individual contributions made by each team member.

Student	Contribution Evidence
Athivaram Jayaprakash (192425311)	Project planning, system architecture, MARL implementation, testing, performance analysis, documentation
K. Anvesh (192472238)	Dataset preparation, smart-city simulation, preprocessing, environment development, simulation testing
G. Vedavikas Gowd (192472123)	RL implementation, algorithm integration, performance evaluation, visualization, testing and documentation

Evidence to Attach

The following evidence may be included where available:

- GitHub/GitLab commit history
- Screenshots of code development
- Simulation outputs
- Training results
- Graph generation
- Meeting photographs/screenshots
- Presentation preparation
- Individual task records
- Testing results
- Documentation contributions

Mandatory for team projects.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Chapter 1, Chapter 2
Engineering design under constraints	SO2	Chapter 3
Written/oral technical communication	SO3	Whole Report + Viva
Ethics and professional responsibility	SO4	Chapter 3 (Section 3.7)
Teamwork and leadership	SO5	Chapter 4 (Section 4.8)
Experimentation, analysis, and engineering judgment	SO6	Chapter 4 (Sections 4.3–4.6)
Independent acquisition of new knowledge	SO7	Chapter 5 (Section 5.3)
Standards	SO2	Chapter 3 (Section 3.2)
Sustainability and societal/environmental impact	SO2 / SO4	Chapter 3 (Section 3.7)
Risk assessment and mitigation	SO2 / SO4	Chapter 3 (Section 3.7)
Security considerations	Relevant SO	Chapter 3 (Section 3.7)
Integrated/system-level engineering	SO1 / SO2	Chapter 3 (Section 3.5)
Project planning and management	SO5	Chapter 4 (Section 4.8)
Individual contribution	SO1–SO7	Individual Contribution Statement + Chapter 4 (Section 4.8)